

Unit 2: Control Statements in Python

2.1 if Statement

The if statement is used for **decision-making** in Python. It allows a program to execute certain code only if a specific condition is True.

Syntax:

if condition:

 # code block

elif another_condition:

 # another code block

else:

 # default code block

Example:

age = 18

if age >= 18:

 print("Eligible to vote")

else:

 print("Not eligible to vote")

2.2 match Statement

The match statement is similar to a switch in other languages. It evaluates an expression and runs code based on matching cases.

Syntax:

match variable:

 case value1:

 # code

```
case value2:
    # code
case _:
    # default case
```

Example:

```
day = "Monday"
match day:
    case "Monday":
        print("Start of the week")
    case "Friday":
        print("End of the week")
    case _:
        print("Midweek day")
```

2.3 break Statement

The break statement **exits the loop** immediately, even if the loop condition is still true.

Example:

```
for i in range(5):
    if i == 3:
        break
    print(i) # Output: 0, 1, 2
```

2.4 continue Statement

The continue statement **skips the current iteration** and moves to the next one.

Example:

```
for i in range(5):  
    if i == 2:  
        continue  
    print(i) # Output: 0, 1, 3, 4
```

2.5 Loop Statements

Loops are used to repeat a block of code multiple times.

2.5.1 while Loop

Repeats a block **as long as** a condition is true.

Syntax:

```
while condition:  
    # code block
```

Example:

```
i = 1  
while i <= 3:  
    print(i)  
    i += 1
```

2.5.2 for Loop

Used to iterate over a **sequence** (like a list, string, or range).

Syntax:

```
for variable in sequence:  
    # code block
```

Example:

```
for i in range(3):
```

```
print(i) # Output: 0, 1, 2
```

2.6 Nested Loops

A loop inside another loop. Useful for solving multi-dimensional problems (e.g., matrices, tables).

Example:

```
for i in range(2):  
    for j in range(3):  
        print(f'i={i}, j={j}')
```

2.7 Infinite Loop

A loop that **never ends** unless a `break` is used or an external condition is changed.

Example:

```
while True:  
    user_input = input("Enter 'exit' to stop: ")  
    if user_input == "exit":  
        break
```